

Project Documentation: Spatial Mapping

Ahmed Elbarbary

Table of Contents

Device Overview	3
Features.....	3
General Description	4
Block Diagram (Data flow graph)	4
Device Characteristics Table	5
Detailed Description	6
Distance Measurement.....	6
Visualization	7
Application Example, Instructions, and Expected output	8
Limitations.....	12
Circuit Schematic	13
Flowchart.....	14

Device Overview

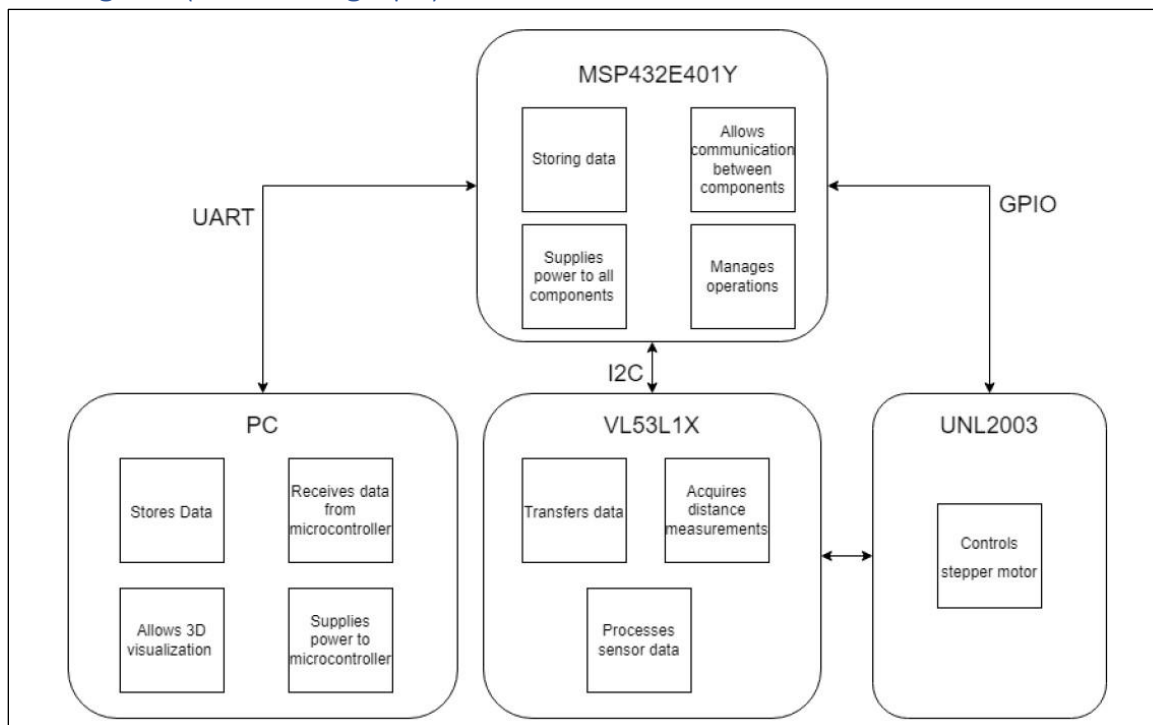
Features

- **Texas Instruments MSP432E401Y Microcontroller**
 - o ARM Cortex-M4F Processor Core
 - o 120 MHz clock speed
 - o Assigned bus speed: 16MHz
 - o Operating Temperature: -40°C – 105°C
 - o Drives the stepper motor and the ToF Sensor
 - o Hardware programmed using Assembly, and C through Keil development environment
 - o Serial communication through Python programming language
 - o 256KB of SRAM
 - o 1024KB of flash memory
 - o 6KB of EEPROM
 - o Two 12-bit ADC modules (max sample rate of 2Msps)
- **ULN2003 Stepper Motor Driver**
 - o 5V–12V operating voltage
 - o 512 steps per rotation (512 steps for every 360°)
 - o Programmed to rotate 1.41° per step for accurate scanning (256 steps = full rotation)
 - o Frequency: 100Hz
 - o LED indicators for 45° rotations
- **VL53L1X Time-of-Flight Sensor**
 - o 2.6V–3.5V operating voltage
 - o Range spanning 400cm of distance
 - o Invisible laser emitter with a wavelength of 940nm
 - o ± 20 mm of ranging error
- **3D Visualization**
 - o Uses PySerial, Numpy, and Open3D libraries
 - o Written in Python 3.10.6
 - o real time plotting using obtained data
- **Data communication**
 - o Baud rate of 115200 bps
 - o UART0 serial communication: between microcontroller and PC
 - o I2C serial communication: between ToF sensor and microcontroller
 - o I2C interface with up to 400kHz speed

General Description

This project is an embedded spatial mapping system that utilizes the VL53L1X time-of-flight (ToF) sensor to collect data about its surroundings. On a low level, this device can be thought of as a transducer, it works by converting a continuous signal to a discrete digital signal. This acquisition of data would not be effective without the sensor rotating to scan the area. To do this, a ULN2003 stepper motor is mounted to the sensor, this provides the system with the ability to obtain a 360° view thanks to the motor's rotation. Information obtained by the time-of-flight sensor is processed, then transmitted as bits to the microcontroller using I2C serial communication. The microcontroller then sends this data to the PC via USB connection by using UART0 communication. Lastly, to complete the 3D visualization, there is a section in the Keil code which calculates the distance measurements using the data from the sensor. Next, the PC utilizes the python code to decode the values generated from Keil, then assigns them as an x, y or z value. From there, an xyz file containing the coordinates is created, this is used to construct a 3D visualization of the system's surroundings. As mentioned, the system is composed of 3 major parts: a time-of-flight sensor (ToF), a microcontroller, and a stepper motor. The MSP432E401Y microcontroller is responsible for handling and managing operations for this embedded system. Before any of the operations are executed, the device undergoes preprocessing to calibrate the system beforehand. This includes things like the bus speed, sensor settings, motor's customizations (step angle, as well as whether it will continuously spin or stop after a certain point).

Block Diagram (Data flow graph)



Device Characteristics Table

Device/Item	Characteristics	Additional Details
Computer and MSP432E401Y Microcontroller	Serial Port	COM5 (UART)
	Bus Speed	16 MHz
	Baud Rate	115200 BPS
Software Tools	Keil uVision5	Assembly and C
	Python (software)	Python 3.10.6
	Python (libraries used)	PySerial, NumPy, Open3D
28BYJ-48 Stepper Motor + ULN2003 Driver	Motor's Pins	Microcontroller's Pins
	+	5V
	-	GND
	IN1	PH0
	IN2	PH1
	IN3	PH2
	IN4	PH3
VL53L1X ToF Sensor	Sensor's Pins	Microcontroller's Pins
	V _{DD}	N/A
	V _{IN}	3.3V
	GND	GND
	SDA	PB3
	SCL	PB2
	XSHUT	N/A
	GPIO1	N/A
Input and Output Devices	Component	Microcontroller Pin
	Push Button 2 (Yellow)	PM0
	Push Button 1 (Black)	PM1
	LED 1 (Measurement Status)	PF4
	LED 2 (Additional Status)	PN1

Detailed Description

Distance Measurement

As previously mentioned, the purpose of the ToF is to scan its surrounding environment in order to collect data about the distance of nearby objects. Here is a detailed description of this process: firstly, the ToF sensor is mounted to the stepper motor using a 3D printed CAD model that holds the sensor in place and attaches to the stepper motor. Next, the motor rotates at approximately 1.41° per step, this means that for a full 360° rotation, there will be 256 measurements performed. The step value was chosen to be small to obtain a more accurate representation of the environment. Once a full rotation has been completed, the user should then move forward x distance (smaller distance will be more accurate) and press the button to scan once more. This repeats until the user is pleased with the number of scans completed. The way distance is measured is as follows: First, the ToF sensor emits infrared light. It determines the distance to the object by measuring the time elapsed for the light to travel to said object and returns. This can be modelled by the following equation:

$$D = \frac{t}{2} * c$$

Where D is the distance being measured, c represents the speed of light, and t represents the infrared's time of travel. Note that t is divided by 2 because it accounts for the fact that the infrared travels there and back. For example, if the infrared sensor takes 2 seconds to reach an object, then 2 seconds to return to the sensor, the time obtained is 4 seconds. This would then be divided by 2 to see that it actually takes 2 seconds to reach that object. Once data is gathered by the sensor, it calculates the distance by converting the time from analog to digital. This is processed to verify that it is within range. If such is the case, the sensor sends this data to the microcontroller using I2C communication.

When receiving the data, the program converts the distance measurements to x, y, and z coordinates. It should be noted that the sensor is only measuring in the yz plane. The displacement on the x-axis is done using the code. Every new scan automatically increments the x value by a specific amount. In other words, when the user moves forward to scan another part of a hallway, the code automatically adds an incrementation to the x coordinate for the 3D visualization to position it forward on the plane. The y coordinate is calculated using the formula: $y = r \cos(\theta)$. As for the z coordinate, it is determined using the formula: $z = r \sin(\theta)$. To illustrate this, here is an example: assume that 300mm was detected as the distance when the first measurement is taken. Firstly, θ would be at 0° , this means the x coordinate would be at 0 because it is only incremented after a full rotation is completed and a new scan is appended. In

this case, we are still scanning the first slice. The y coordinate would be $y = 300\cos(0) = 300$. And the z coordinate would be $z = 300\sin(0) = 0$. The next measurement would be at 1.41 degrees. Note that x will still be 0 because it is still the same slice being scanned. The y and z coordinates however will change because θ is now 1.41 instead of 0. This process repeats for all 256 steps in the rotation. After a full rotation is completed, when another scan slice begins, the x coordinate will then change as mentioned earlier at the beginning of this explanation.

Visualization

The visualization of the environment was done through the use of python alongside the ToF sensor. While the sensor collects the data, the PySerial library was used to assist with the communication. In terms of the actual 3D visualization, two libraries played a major role: Open3D, and NumPy. The Open3D library provided functions for point cloud (PCD) visualization. An example function is `o3d.io.read_point_cloud()` which was important for loading and reading the point cloud file. `o3d.visualization.draw_geometries()` is the function used to display the point cloud. In addition, *Lineset* was used to create line geometries. The way this works is that the points along the y axis are connected. For this to occur, a for loop is utilized in the code and lines are created by appending the points to an array. The following figure illustrates how this was done in python.

```

117
118 for a in range(0,num_levels*num_points,256):
119     for i in range(256): ##loop through all the points
120         if i==255: #if we are on the last point, append it or connect it to the first one
121             line_list.append([a+i,a])
122         else:
123             line_list.append([a+i,a+i+1]) ##if we are on any other point, connect it to the next one
124
125 for a in range(0,255): ##same thing but with the levels
126     for i in range(1,num_levels):
127         line_list.append([a+(i-1)*256,a+i*256])
128

```

Lastly, the line colors are customized according to preference, then the 3D visualization is plotted using the following function: `o3d.visualization.draw_geometries()`.

Application Example, Instructions, and Expected output

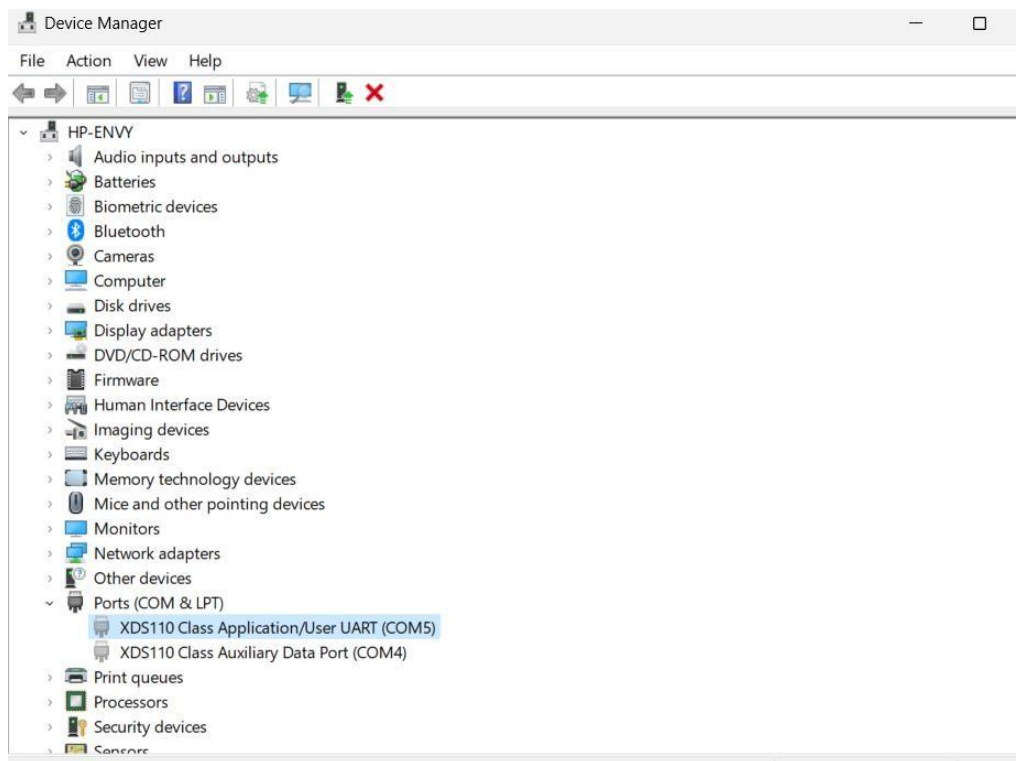
To use this embedded system effectively, simply follow the following instructions. First, set up the environment and download the required software by following these steps. It is important to ensure that these steps are adhered to in order for the system to work.

1. Download, and install KEIL uVision5 using the link: <https://www2.keil.com/mdk5> and download MDK. Once the installation is complete, you will see the pack installer. Click the “Devices” tab, and search for MSP432E401Y. Ensure that you Install the family pack.
2. Download and install Python 3.10.6 or newer using <https://www.python.org/downloads/>. After running the set up and installing python, ensure that you have added it to the path.
3. Once Python has been installed, it is recommended that you download any IDE that can run python. I used VS code and strongly recommend it.
4. Now, it is time to install all the important python libraries that are important for the serial communication and the 3D visualization. Simply follow the following instructions open command prompt in administrator mode by right clicking “Command Prompt” and then clicking “Run as administrator.”

Once the command prompt is running, you will need to install PySerial, NumPy, and Open3D. To do this, enter the following commands: “pip install pyserial.” Wait for a confirmation message confirming that packages have been installed. Next, complete the same process for “python -m pip install -U numpy,” and “python -m pip install -U Open3D” to install NumPy and Open3D respectively.

At this point, you have all the necessary software components installed, now, it is time to run the provided code and see your embedded system in action:

1. Download the provided zip folder and make sure to extract it in a folder of your preference. Scroll down to the file named: *Deliverable_3D*. Using the IDE of your choice, open the file in edit mode and follow the following instructions carefully:
 - a. In line 35, There is a line that says: `s = serial.Serial('COM5',115200,timeout =` Firstly, you may change the baud rate by editing the number “115200”. In addition, make sure to Change the COM port to the designated port on your computer. To determine your UART COM port, go to “Device Manager”. Scroll down to “Ports” and you will see your COM port can be found listed next to “XDS110 Class Application/User UART”. Make sure that your microcontroller is connected during this step. The following figure illustrates this.



b. Next, open the Keil file with the name: *2dx_studio_8c*, there are a few customizations that can be made. On line 131, you may change *z* to your desired step value, it is currently at 2 which is equivalent to 1.41° .

2. Ensure your microcontroller and circuit are connected to the PC via the micro-USB port. Remember to use the port on the opposite side of the ethernet port

3. Once ready, click “Translate”, “Build”, and “Download” in Keil. Then click the reset button on the micro controller

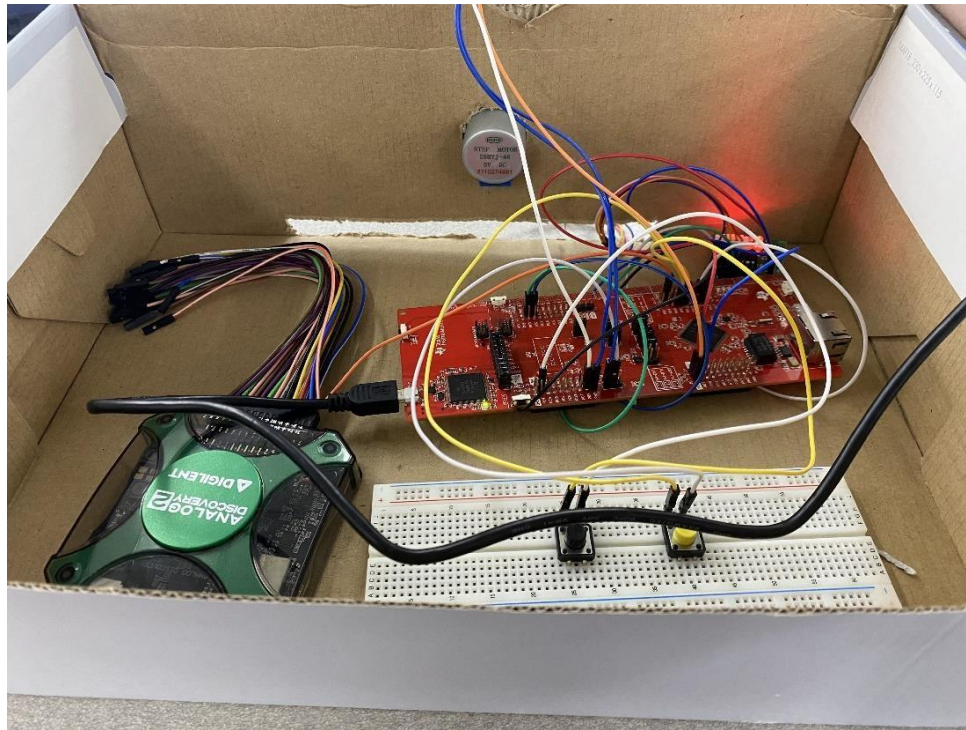
4. Open the file in KEIL and then click the “Translate”, “Build”, and “Download” buttons in that order. Once that is complete, click the reset button which is next to the micro-USB port on the microcontroller.

5. Return to the previously opened python file, and run the program. At this point, press “Enter” on your keyboard, the press button 2 (connected to PM0). At this point your sensor is initialized and ready to scan at your discretion.

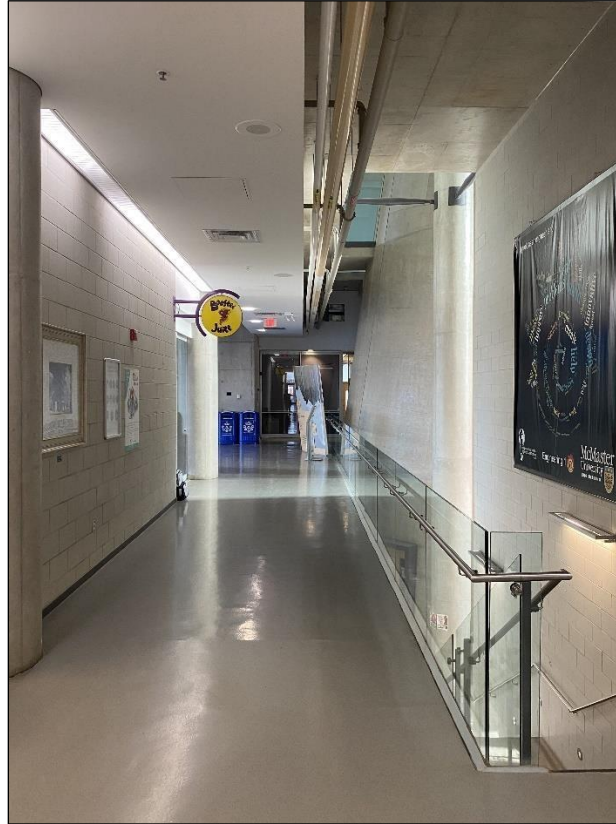
6. Whenever ready, press button 1 (connected to PM1), and at this point, the motor will start spinning with the sensor scanning the environment. Once a full rotation has been

completed, you can either start another scan, or stop if you are not interested in scanning more slices. To scan another slice, press the button connected to PM1 again. You may repeat this process however many times you like. Once you are happy with the result, press the button connected to PM0 to terminate the scanning procedure and begin plotting the scanned data using open 3D. Note that Open3D will open automatically and display the plot at this point.

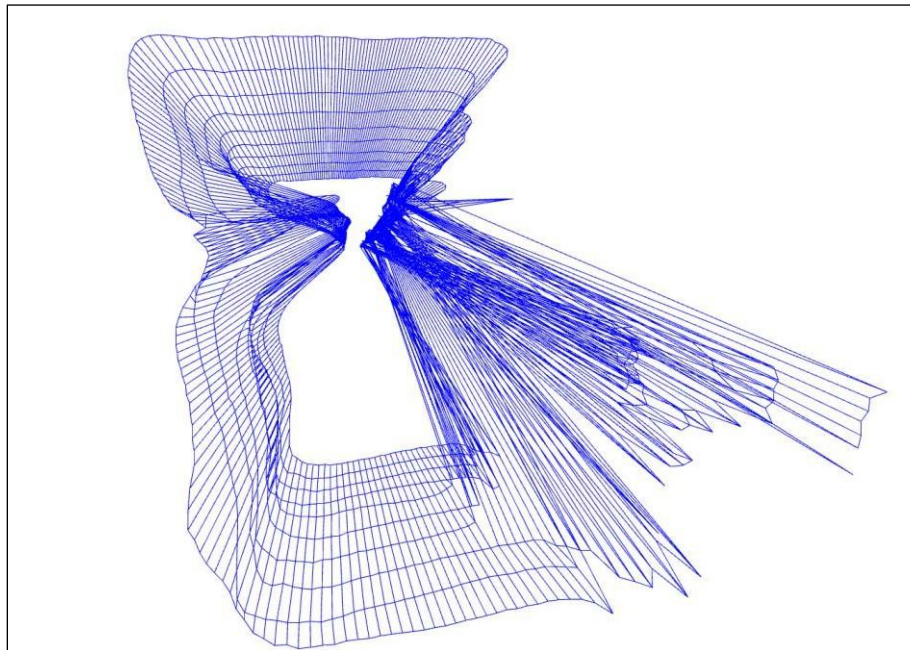
The following is an example of applying these instructions:



In this figure, the yellow button is connected to PM0 and the black button is connected to PM1.



Picture of hallway that was scanned using this system



Generated Image after scanning 10 slices while walking through this hallway.

Limitations

1. limitations of the microcontroller floating point capability and use of trigonometric functions.

The microcontroller can carry out single precision mathematical operations because it uses floating point. This is why we use the math library to obtain the x, y, and z coordinates.

2. maximum quantization error for each of the ToF module.

The maximum quantization error is equal to $V_{fs}/2^n = 4V/2^{16} = 6.1 \cdot 10^{-5}$

3. the maximum standard serial communication rate you implemented with the PC.

The maximum standard serial communication rate that can be implemented with the PC is 115200 bits/sec. This was the baudrate used in UART and it was verified with real term.

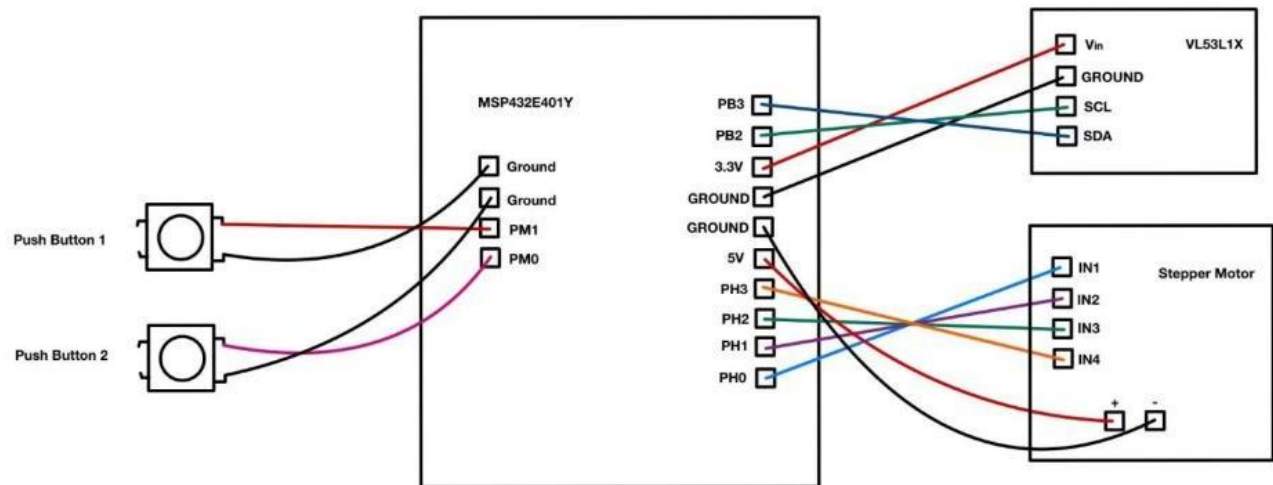
4. the communication method(s) and speed used between the microcontroller and the ToF modules.

The communication method used between the microcontroller and the Time-of-Flight sensor was I2C communication protocol as mentioned earlier in the report. The microcontroller is receiving the data and it acts as the master in this architecture. The sensor on the other hand is a slave peripheral and it sends the data to the microcontroller. The speed used between them is 100k bits per second.

5. the primary limitation on system speed.

The ToF sensor was a primary limitation on the system. As seen through my scan of the hall way, one wall looks perfectly modeled, while the other one is extremely messy due to the fact that was far. The sensor was not able to smoothly display that part of the hall way. In addition, it was not possible to shorten the delay in this system as it would make the sensor scan even less accurate. On the other hand if it is increased, the scanning would be extremely slow, as a result it places a restriction on it and makes it the primary limitation on the system.

Circuit Schematic



Flowchart

